RÉPUBLIQUE DU CAMEROUN
Paix – Travail – Patrie **REPUBLIC OF CAMEROON**
Peace – Work – Fatherland

ÉCOLE NATIONALE SUPÉRIEURE POLYTECHNIQUE
DE YAOUNDÉ
Département Génie Informatique
classe : AIA4

Rapport de Travaux Pratiques

Réalité Augmentée From Scratch

Bibliothèque mathématique NkMath
Semaines 1 à 4 — Modules 01 à 04

Informations de l'étudiant

Nom & Prénom :	KENNE TAKOUDJOU Franck Manuel
Classe :	AIA4
Établissement :	ENSPY – Yaoundé
Cours :	Réalité Augmentée From Scratch
Enseignant :	TEUGUIA TADJUIDJE Rodolf Sédéris
Année académique :	2025 – 2026

Technologies utilisées : C++17 • Jenga Build System • Nkentseu Engine • GitHub

Yaoundé, Mars 2026

Contents

Introduction Générale **3**

Semaine 1 — IEEE 754 & Arithmétique Flottante **4**
Objectifs de la semaine 4
0.1 Théorie : Représentation IEEE 754 4
0.1.1 Structure d'un float 32 bits 4
0.1.2 Valeurs spéciales 4
0.1.3 L'epsilon machine 5
0.2 TP1 — inspectFloat (Tests 1 à 6) 5
0.2.1 Objectif 5
0.2.2 Implémentation 5
0.2.3 Résultats et explications 6
0.3 TP2 — Kahan et Welford (Tests 7 à 10) 6
0.3.1 Objectif 6
0.3.2 Sommation de Kahan 6
0.3.3 Variance de Welford 6
0.3.4 Résultats 7
0.4 TP3 — Float.h (Tests 11 à 14) 7
0.4.1 Objectif 7
0.4.2 Implémentation de Float.h 7
0.4.3 Tests unitaires (extrait) 8

Semaine 2 — Vecteurs : Vec2d, Vec3d, Vec4d **9**
Objectifs de la semaine 9
0.5 TP1 — Vec2d.h (Tests 15 à 19) 9
0.5.1 Objectif 9
0.5.2 Implémentation 9
0.5.3 Tests (15 à 19) 10
0.6 TP2 — Vec3d.h + Gram-Schmidt (Tests 20 à 23) 10
0.6.1 Objectif 11
0.6.2 Produit vectoriel 3D 11
0.6.3 Gram-Schmidt 11
0.6.4 Tests (20 à 23) 11
0.7 TP3 — Vec4d.h + Projection (Tests 24 à 27) 12
0.7.1 Objectif 12
0.7.2 Coordonnées homogènes 12
0.7.3 Projection perspective 12

Semaine 3 — Matrices : Mat4d, TRS, Rasteriseur **13**
Objectifs de la semaine . . . 13
0.8 TP1 — Mat4d.h + Inverse (Tests 28 à 31) . . . 13
0.8.1 Objectif . . . 13
0.8.2 Structure Mat4d . . . 13
0.8.3 Inverse Gauss-Jordan . . . 14
0.8.4 Rotation de Rodrigues . . . 14
0.8.5 Tests (28 à 31) . . . 15
0.9 TP2 — trs-decomposition.cpp (Tests 38 à 40) . . . 15
0.9.1 Objectif . . . 15
0.9.2 Construction TRS . . . 15
0.9.3 Décomposition inverse . . . 16
0.10 TP3 — rasterizer-lookat.cpp (Tests 32 à 37) . . . 16
0.10.1 Objectif . . . 16
0.10.2 Matrice LookAt . . . 16
0.10.3 Boucle de rendu . . . 17

Semaine 4 — Quaternions & Animation SLERP **18**
Objectifs de la semaine . . . 18
0.11 Théorie : Quaternions . . . 18
0.11.1 Définition . . . 18
0.11.2 Produit de Hamilton . . . 18
0.11.3 SLERP — Spherical Linear Interpolation . . . 18
0.12 TP1 — Quaternions complets (Tests 41 à 43) . . . 19
0.12.1 Objectif . . . 19
0.12.2 Implémentation de Quat.h . . . 19
0.12.3 Méthode de Shepperd — ToMat3 / FromMat3 . . . 20
0.12.4 Tests (41 à 43) . . . 20
0.13 TP2 — Animation SLERP (Tests 44 à 46) . . . 21
0.13.1 Objectif . . . 21
0.13.2 Implémentation SLERP . . . 21
0.13.3 Comparaison SLERP vs LERP . . . 22
0.13.4 Tests (44 à 46) . . . 22

Conclusion Générale **24**
Bilan des travaux pratiques . . . 24
Perspectives . . . 25

Introduction

Ce rapport présente les travaux pratiques réalisés dans le cadre du cours **Réalité Augmentée From Scratch** dispensé à l'École Nationale Supérieure Polytechnique de Yaoundé.

L'objectif de ce cours est de construire un pipeline complet de réalité augmentée en **C++17**, intégré dans le moteur **Nkentseu**, sans aucune bibliothèque tierce (pas d'OpenCV, pas de GLM, pas d'Eigen). Le build system utilisé est **Jenga**, développé par l'équipe Rihen.

Règle absolue du cours

Zéro bibliothèque tierce. Uniquement la STL C++17 et les API système natives. Toute soumission contenant un `#include` d'une bibliothèque interdite est invalidée automatiquement.

Ce rapport couvre les **4 premières semaines** du semestre 1, soit **11 travaux pratiques** portant sur la bibliothèque mathématique **NkMath** :

- **Semaine 1** : IEEE 754, arithmétique flottante (TP1, TP2, TP3)
- **Semaine 2** : Vecteurs Vec2d, Vec3d, Vec4d (TP1, TP2, TP3)
- **Semaine 3** : Matrices Mat4d, TRS, Rasteriseur (TP1, TP2, TP3)
- **Semaine 4** : Quaternions, SLERP, Animation (TP1, TP2)

Semaine 1 — IEEE 754 & Arithmétique Flottante

Objectifs de la semaine

Pourquoi ce module est indispensable

Tout le pipeline AR manipule des nombres flottants : coordonnées 3D, matrices de projection, résidus de calibration, valeurs de pixels. Une erreur d'arrondi invisible dans la résolution d'un système linéaire peut faire diverger Levenberg-Marquardt, produire une homographie incorrecte, ou générer une pose décalée de plusieurs centimètres. Ce module est la **fondation de tout code numérique fiable**.

0.1 Théorie : Représentation IEEE 754

0.1.1 Structure d'un float 32 bits

Un nombre flottant IEEE 754 de 32 bits est découpé en trois champs :

$$\text{Valeur} = (-1)^{\text{signe}} \times 2^{\text{exposant}-127} \times 1.\text{mantisse} \tag{1}$$

Bit 31	**Bits 30–23**	**Bits 22–0**
Signe (1 bit) 0 = positif	Exposant biaisé (8 bits) biais = 127	Mantisse (23 bits) bit implicite 1. devant

Table 1: Structure d'un float IEEE 754 32 bits

0.1.2 Valeurs spéciales

Type	**Signe**	**Exposant**	**Mantisse**
+0	0	0x00	0x000000
-0	1	0x00	0x000000
+Inf	0	0xFF	0x000000
-Inf	1	0xFF	0x000000
NaN	X	0xFF	$\neq 0$
Subnormal	X	0x00	$\neq 0$

Table 2: Valeurs spéciales IEEE 754

Piège classique : NaN

`NaN != NaN` est **VRAI** — la seule façon de tester NaN est `std::isnan(x)`. Jamais utiliser `==` avec NaN.

0.1.3 L'epsilon machine

L'epsilon machine (ε) est la plus petite valeur positive telle que $1.0 + \varepsilon \neq 1.0$.

- Pour `float` : $\varepsilon \approx 1.19 \times 10^{-7}$
- Pour `double` : $\varepsilon \approx 2.22 \times 10^{-16}$

0.2 TP1 — inspectFloat (Tests 1 à 6)

0.2.1 Objectif

Implémenter la fonction `inspectFloat(float x)` qui affiche les 3 champs (signe, exposant, mantisse) en binaire sur la console.

0.2.2 Implémentation

```cpp
// Lecture bits IEEE 754 -- C++17 legal (memcpy, pas de UB)
static uint32_t floatBits(float f) {
    uint32_t bits;
    std::memcpy(&bits, &f, sizeof(float));
    return bits;
}

void inspectFloat(float f) {
    uint32_t bits     = floatBits(f);
    uint32_t sign     = (bits >> 31) & 0x1;
    uint32_t exponent = (bits >> 23) & 0xFF;
    uint32_t mantissa =  bits        & 0x7FFFFF;

    printf("float = %.10f | hex = 0x%08X\n", f, bits);
    printf("  signe    = %u\n", sign);
    printf("  exposant = %u (2^%d)\n",
           exponent, (int)exponent - 127);
    printf("  mantisse = 0x%06X\n", mantissa);

    if      (exponent == 0xFF && mantissa != 0)
        printf("  type = NaN\n");
    else if (exponent == 0xFF && mantissa == 0)
        printf("  type = %cInf\n", sign ? '-' : '+');
    else if (exponent == 0x00 && mantissa != 0)
        printf("  type = Subnormal\n");
    else if (exponent == 0x00 && mantissa == 0)
        printf("  type = %c0\n", sign ? '-' : '+');
    else
        printf("  type = Normal\n");
}
```

Listing 1: Float.cpp — inspectFloat

0.2.3 Résultats et explications

Valeur	Type	Explication
`0.1f`	Normal	Pas exactement représentable en binaire
`1.0f`	Normal	signe=0, exposant=127, mantisse=0
`1.0f/0.0f`	+Inf	Exposant=0xFF, mantisse=0
`sqrt(-1.0f)`	NaN	Exposant=0xFF, mantisse≠0
`-0.0f`	-0	Signe=1, exposant=0, mantisse=0
`FLT_MIN`	Subnormal	Exposant=0x00, mantisse≠0

Table 3: Résultats de inspectFloat sur les 6 valeurs

Pourquoi 0.1f n'est pas exact ?

En base 2, 0.1 en décimal est une fraction périodique infinie : $0.1_{10} = 0.0\overline{0011}_2$. Il est donc impossible de le représenter exactement avec 23 bits de mantisse. La valeur stockée est $\approx 0.10000000149011612$.

0.3 TP2 — Kahan et Welford (Tests 7 à 10)

0.3.1 Objectif

Démontrer empiriquement les problèmes de précision numérique : sommation de Kahan vs `std::accumulate`, et variance de Welford vs variance naïve.

0.3.2 Sommation de Kahan

L'algorithme de Kahan compense les erreurs d'arrondi accumulées lors de la somme de nombreux termes.

```
float kahanSum(const float* data, int n) {
    float sum  = 0.0f;
    float comp = 0.0f; // compensation des erreurs perdues
    for (int i = 0; i < n; i++) {
        float y = data[i] - comp; // compenser l'erreur precedente
        float t = sum + y;        // t grand, y petit -> perte bits
        comp = (t - sum) - y;     // capture les bits perdus
        sum  = t;
    }
    return sum;
}
```

Listing 2: Float.cpp — kahanSum

0.3.3 Variance de Welford

```cpp
float varianceWelford(const float* data, int n) {
    float mean = 0.0f;
    float M2   = 0.0f;
    for (int i = 0; i < n; i++) {
        float delta  = data[i] - mean;
        mean        += delta / (i + 1);
        float delta2 = data[i] - mean;
        M2          += delta * delta2;
    }
    return M2 / (n - 1);
}
```

Listing 3: Float.cpp — varianceWelford

0.3.4 Résultats

Méthode	Résultat	Erreur
Valeur théorique	100000.0000	0.0000
`std::accumulate`	$\approx$ 100006.xxxx	$\approx$ 6.xxxx
`kahanSum`	$\approx$ 100000.0000	$\approx$ 0.0000

Table 4: Test 7-8 : Somme de 1 000 000 $\times$ 0.1f

Cancellation catastrophique

La variance naïve de $\{10^8, 10^8, 1, 2\}$ peut donner un résultat **négatif** ! C'est la cancellation catastrophique : on soustrait deux très grands nombres presque égaux. La méthode de Welford est numériquement stable car elle ne calcule jamais sum^2.

0.4 TP3 — Float.h (Tests 11 à 14)

0.4.1 Objectif

Créer le fichier `Float.h` avec toutes les fonctions utilitaires et écrire 30 tests unitaires minimum.

0.4.2 Implémentation de Float.h

```cpp
namespace NkMath {

constexpr double kEps  = 1e-9;
constexpr float  kFEps = 1e-6f;

// Valide : ni NaN, ni Inf
inline bool isFiniteValid(float  x) { return std::isfinite(x); }
inline bool isFiniteValid(double x) { return std::isfinite(x); }

// Proche de zero
inline bool nearlyZero(float  x, float  eps = kFEps) {
    return std::abs(x) < eps;
```

```
}
inline bool nearlyZero(double x, double eps = kEps) {
    return std::abs(x) < eps;
}

// Egalite relative
inline bool approxEq(double a, double b, double eps = kEps) {
    if (a == b) return true;
    double maxAB = std::max(std::abs(a), std::abs(b));
    return std::abs(a - b) <= eps * std::max(1.0, maxAB);
}

// Sommation de Kahan
double kahanSum(const double* data, int n);

} // namespace NkMath
```

Listing 4: Float.h — fonctions utilitaires

0.4.3 Tests unitaires (extrait)

```
// Test 11 : isFiniteValid
assert(!isFiniteValid(std::sqrt(-1.0f)));  // NaN invalide
assert(!isFiniteValid(1.0f / 0.0f));       // +Inf invalide
assert( isFiniteValid(1.0f));              // Normal valide

// Test 12 : nearlyZero
assert( nearlyZero(0.0f));
assert( nearlyZero(1e-7f));
assert(!nearlyZero(0.1f));

// Test 13 : approxEq
assert( approxEq(1.0f, 1.0f));
assert( approxEq(1.0f, 1.0f + 1e-7f));
assert(!approxEq(1.0f, 1.1f));
```

Listing 5: Tests Float.h

Semaine 2 — Vecteurs : Vec2d, Vec3d, Vec4d

Objectifs de la semaine

Pourquoi les vecteurs sont l'alphabet du pipeline 3D

Les vecteurs sont la structure de données la plus utilisée du pipeline AR. Chaque position dans l'espace est un Vec3d. Chaque pixel a une position Vec2d. Chaque coordonnée homogène transmise à la matrice de projection est un Vec4d. Sans une implémentation rigoureuse avec `static_assert` sur `sizeof`, rien d'autre dans NkMath ne fonctionne.

0.5 TP1 — Vec2d.h (Tests 15 à 19)

0.5.1 Objectif

Implémenter `Vec2d` avec tous les opérateurs, le dot product, le cross product 2D, et la normalisation.

0.5.2 Implémentation

```cpp
struct Vec2d {
    double x, y;

    Vec2d() : x(0.0), y(0.0) {}
    Vec2d(double x, double y) : x(x), y(y) {}
    explicit Vec2d(double s) : x(s), y(s) {} // fill constructor

    // Acces par index -- legal C++17 standard-layout
    double& operator[](int i) {
        assert(i >= 0 && i < 2 && "Vec2d index out of bounds");
        return (&x)[i]; // garanti contigu en C++17
    }

    // Operateurs arithmetiques
    Vec2d operator+(const Vec2d& o) const { return {x+o.x, y+o.y}; }
    Vec2d operator-(const Vec2d& o) const { return {x-o.x, y-o.y}; }
    Vec2d operator*(double s)       const { return {x*s,   y*s  }; }
    Vec2d operator/(double s)       const { return {x/s,   y/s  }; }

```

```
    // Norme et normalisation
    double Norm2() const { return x*x + y*y; }
    double Norm()  const { return std::sqrt(Norm2()); }
    Vec2d Normalized() const {
        double n = Norm();
        if (nearlyZero(n)) return Vec2d(0.0); // jamais NaN
        return {x/n, y/n};
    }
};

// Dot product : a.b = |a||b|cos(theta)
inline double Dot(const Vec2d& a, const Vec2d& b) {
    return a.x*b.x + a.y*b.y;
}

// Cross 2D (scalaire)
inline double Cross2D(const Vec2d& a, const Vec2d& b) {
    return a.x*b.y - a.y*b.x;
}

// Garantie layout memoire pour glVertexAttribPointer
static_assert(sizeof(Vec2d) == 16, "Vec2d must be 16 bytes");
static_assert(offsetof(Vec2d, x) == 0, "x must be first");
static_assert(offsetof(Vec2d, y) == 8, "y must be at offset 8");
```

Listing 6: Vec2d.h — structure complete

0.5.3 Tests (15 à 19)

```
// Test 15 : Dot product
assert(approxEq(Dot(Vec2d(1,0), Vec2d(0,1)),  0.0));
assert(approxEq(Dot(Vec2d(1,0), Vec2d(1,0)),  1.0));
assert(approxEq(Dot(Vec2d(3,4), Vec2d(3,4)), 25.0));

// Test 16 : Cross2D
assert(approxEq(Cross2D(Vec2d(1,0), Vec2d(0,1)),  1.0));
assert(approxEq(Cross2D(Vec2d(0,1), Vec2d(1,0)), -1.0));

// Test 17 : Normalisation
Vec2d n = Vec2d(3.0, 4.0).Normalized();
assert(approxEq(n.Norm(), 1.0, kEps));

// Test 18 : Operateur []
Vec2d v(5.0, 7.0);
assert(approxEq(v[0], 5.0));
v[0] = 99.0;
assert(approxEq(v.x, 99.0));

// Test 19 : Layout memoire
static_assert(sizeof(Vec2d) == 16, "Vec2d 16 bytes");
```

Listing 7: Tests Vec2d

0.6 TP2 — Vec3d.h + Gram-Schmidt (Tests 20 à 23)

0.6.1 Objectif

Implémenter `Vec3d` avec le produit vectoriel 3D et l'orthogonalisation de Gram-Schmidt.

0.6.2 Produit vectoriel 3D

$$\vec{a} \times \vec{b} = \begin{vmatrix} \vec{i} & \vec{j} & \vec{k} \\ a_x & a_y & a_z \\ b_x & b_y & b_z \end{vmatrix} = (a_y b_z - a_z b_y,\ a_z b_x - a_x b_z,\ a_x b_y - a_y b_x) \tag{2}$$

```
// Cross product 3D -- regle main droite
// Mnemonique permutations cycliques : x->y->z->x
inline Vec3d Cross(const Vec3d& a, const Vec3d& b) {
    return {
        a.y*b.z - a.z*b.y,  // composante x
        a.z*b.x - a.x*b.z,  // composante y
        a.x*b.y - a.y*b.x   // composante z
    };
}
```

Listing 8: Vec3d.h — Cross product

0.6.3 Gram-Schmidt

```
struct OrthoBasis { Vec3d u, v, w; };

inline OrthoBasis GramSchmidt(Vec3d a, Vec3d b, Vec3d c) {
    // Etape 1 : normaliser a -> u
    Vec3d u = a.Normalized();
    // Etape 2 : oter la composante u de b -> v
    Vec3d v = (b - Project(b, u)).Normalized();
    // Etape 3 : oter les composantes u et v de c -> w
    Vec3d w = (c - Project(c, u) - Project(c, v)).Normalized();
    return {u, v, w};
}
```

Listing 9: Vec3d.h — GramSchmidt

0.6.4 Tests (20 à 23)

```
// Test 20 : Cross regle main droite
Vec3d r = Cross(Vec3d(1,0,0), Vec3d(0,1,0));
assert(approxEq(r.z, 1.0)); // X x Y = Z

// Test 21 : Cross non-commutatif
Vec3d r2 = Cross(Vec3d(0,1,0), Vec3d(1,0,0));
assert(approxEq(r2.z, -1.0)); // Y x X = -Z

// Test 22 : Gram-Schmidt -- 10 triplets aleatoires
OrthoBasis gs = GramSchmidt(a, b, c);
assert(approxEq(gs.u.Norm(), 1.0, 1e-9));
assert(approxEq(Dot(gs.u, gs.v), 0.0, 1e-9)); // orthogonaux

// Test 23 : Project + Reject = a
```

```
Vec3d proj = Project(a, b);
Vec3d rej  = Reject(a, b);
assert((proj + rej) == a); // recomposition exacte
```

Listing 10: Tests Vec3d

0.7 TP3 — Vec4d.h + Projection (Tests 24 à 27)

0.7.1 Objectif

Implémenter `Vec4d` avec la déhomogénéisation, projeter les 8 coins d'un cube, et générer une image PPM.

0.7.2 Coordonnées homogènes

- $w = 1.0 \rightarrow$ **POINT** dans l'espace (la translation de Mat4 s'applique)
- $w = 0.0 \rightarrow$ **DIRECTION** (la translation ne s'applique **pas**)

0.7.3 Projection perspective

$$u = f_x \cdot \frac{X}{Z} + c_x \qquad v = f_y \cdot \frac{Y}{Z} + c_y \tag{3}$$

Avec $f_x = f_y = 500$, $c_x = c_y = 256$, distance caméra $z_{cam} = 2.0$.

```
// Test 25 : Projection simple
for (int i = 0; i < 8; i++) {
    double u = fx * (coins_cam[i].x / coins_cam[i].z) + cx;
    double v = fy * (coins_cam[i].y / coins_cam[i].z) + cy;
    assert(u >= 0 && u < 512 && v >= 0 && v < 512);
}

// Test 26 : NkImage 512x512 -- afficher les 8 coins
NkImage img(512, 512);
img.Fill(0, 0, 0);
for (int i = 0; i < 8; i++)
    img.DrawPoint(px[i], py[i], 255, 0, 0); // rouge

// Test 27 : 12 aretes en vert
for (int i = 0; i < 12; i++)
    img.DrawLine(px[a], py[a], px[b], py[b], 0, 255, 0);

img.SavePPM("cube_projection.ppm");
```

Listing 11: Projection perspective simple

Format PPM P6

Le format PPM (Portable Pixmap) P6 est un format d'image binaire sans dépendance externe. Structure : `P6\n largeur hauteur\n 255\n` suivi des pixels RGB. C'est le format idéal pour vérifier visuellement les résultats sans bibliothèque d'images.

Semaine 3 — Matrices : Mat4d, TRS, Rasteriseur

Objectifs de la semaine

Convention column-major — critique pour OpenGL

En mathématiques, on note les matrices en row-major (ligne par ligne). OpenGL utilise la convention **column-major** (colonne par colonne). Cette différence est fondamentale : une matrice transposée par erreur dans l'upload GPU produit des transformations incorrectes.

```
data[col*4 + row] = m(row, col)
```

0.8 TP1 — Mat4d.h + Inverse (Tests 28 à 31)

0.8.1 Objectif

Implémenter `Mat4d` avec produit matriciel et inverse Gauss-Jordan avec pivot partiel.

0.8.2 Structure Mat4d

```
struct Mat4d {
    // Stockage column-major : data[col*4 + row]
    double data[16];

    // Acces at(row, col)
    double& operator()(int row, int col) {
        return data[col*4 + row]; // column-major !
    }

    // Identite
    static Mat4d Identity() {
        Mat4d m;
        m(0,0) = m(1,1) = m(2,2) = m(3,3) = 1.0;
        return m;
    }

    // Produit matriciel O(64 multiplications pour 4x4)
    Mat4d operator*(const Mat4d& o) const {
        Mat4d result = Mat4d::Zero();
        for (int row = 0; row < 4; row++)
```

```
            for (int col = 0; col < 4; col++) {
                double sum = 0.0;
                for (int k = 0; k < 4; k++)
                    sum += (*this)(row,k) * o(k,col);
                result(row,col) = sum;
            }
        return result;
    }
};
```

Listing 12: Mat4d.h — structure column-major

0.8.3 Inverse Gauss-Jordan

```
bool Inverse(Mat4d& out) const {
    // Matrice augmentee [M | I]
    double aug[4][8];
    for (int r = 0; r < 4; r++)
        for (int c = 0; c < 4; c++) {
            aug[r][c]   = (*this)(r,c);
            aug[r][c+4] = (r == c) ? 1.0 : 0.0;
        }

    for (int col = 0; col < 4; col++) {
        // Pivot partiel : trouver le max absolu
        int pivotRow = col;
        for (int r = col+1; r < 4; r++)
            if (abs(aug[r][col]) > abs(aug[pivotRow][col]))
                pivotRow = r;

        if (nearlyZero(aug[col][col])) return false; // singuliere

        // Normaliser et eliminer
        // ...
    }
    return true;
}
```

Listing 13: Mat4d — Inverse Gauss-Jordan

0.8.4 Rotation de Rodrigues

$$R = I\cos\theta + (1-\cos\theta)\,\hat{n}\hat{n}^T + \sin\theta\,[\hat{n}]_\times \tag{4}$$

```
static Mat4d RotateAxis(const Vec3d& axis, double angleRad) {
    Vec3d n = axis.Normalized();
    double c = std::cos(angleRad);
    double s = std::sin(angleRad);
    double t = 1.0 - c;

    Mat4d R = Mat4d::Identity();
    R(0,0) = t*n.x*n.x + c;
    R(0,1) = t*n.x*n.y - s*n.z;
    R(0,2) = t*n.x*n.z + s*n.y;
    // ...
```

```
    return R;
}
```

Listing 14: Mat4d — RotateAxis Rodrigues

0.8.5 Tests (28 à 31)

```
// Test 28 : M x Identity = M pour 10 matrices aleatoires
assert((M * I) == M);
assert((I * M) == M);

// Test 29 : M x   M     = I a 1e-10
Mat4d Inv; M.Inverse(Inv);
Mat4d MInv = M * Inv;
for (int r = 0; r < 4; r++)
    for (int c = 0; c < 4; c++) {
        double expected = (r == c) ? 1.0 : 0.0;
        assert(std::abs(MInv(r,c) - expected) < 1e-10);
    }

// Test 30 : matrice singuliere retourne false
Mat4d S = Mat4d::Zero(); S(0,0) = 1.0; S(1,1) = 1.0;
Mat4d inv;
assert(!S.Inverse(inv)); // false = singuliere

// Test 31 : RotateAxis(Y, PI/2) x {1,0,0,1} = {0,0,-1,1}
Mat4d R = Mat4d::RotateAxis(Vec3d(0,1,0), kPi/2.0);
Vec4d v(1,0,0,1);
Vec4d r = R * v;
assert(approxEq(r.x,  0.0, 1e-10));
assert(approxEq(r.z, -1.0, 1e-10));
```

Listing 15: Tests Mat4d

0.9 TP2 — trs-decomposition.cpp (Tests 38 à 40)

0.9.1 Objectif

Implémenter TRS complet et vérifier la décomposition inverse pour 20 triplets (T, R, S) aléatoires.

0.9.2 Construction TRS

$$M_{TRS} = T \cdot R \cdot S \tag{5}$$

L'**ordre est crucial** : scale d'abord, puis rotation, puis translation.

```
static Mat4d TRS(const Vec3d& t, const Vec3d& axis,
                 double angle, const Vec3d& s) {
    return Translate(t) * RotateAxis(axis, angle) * Scale(s);
}
```

Listing 16: Mat4d — TRS

0.9.3 Décomposition inverse

```
TRSResult decomposeTRS(const Mat4d& M) {
    TRSResult res;

    // Etape 1 : Translation = colonne 3
    res.translation = Vec3d(M(0,3), M(1,3), M(2,3));

    // Etape 2 : Scale = norme de chaque colonne 0,1,2
    double sx = sqrt(M(0,0)*M(0,0)+M(1,0)*M(1,0)+M(2,0)*M(2,0));
    double sy = sqrt(M(0,1)*M(0,1)+M(1,1)*M(1,1)+M(2,1)*M(2,1));
    double sz = sqrt(M(0,2)*M(0,2)+M(1,2)*M(1,2)+M(2,2)*M(2,2));
    res.scale = Vec3d(sx, sy, sz);

    // Etape 4 : Angle depuis trace(R) = 1 + 2*cos(angle)
    double trace = res.rotation(0,0)+res.rotation(1,1)+res.rotation(2,2)
   ;
    res.angleRad = std::acos((trace - 1.0) * 0.5);

    return res;
}
```

Listing 17: trs-decomposition.cpp — decomposeTRS

0.10 TP3 — rasterizer-lookat.cpp (Tests 32 à 37)

0.10.1 Objectif

Implémenter un rasteriseur logiciel dans une `NkImage` 512×512, afficher un cube en rotation sur 10 frames.

0.10.2 Matrice LookAt

```
static Mat4d LookAt(const Vec3d& eye,
                    const Vec3d& target,
                    const Vec3d& worldUp) {
    Vec3d f = (target - eye).Normalized(); // forward
    Vec3d r = Cross(f, worldUp).Normalized(); // right
    Vec3d u = Cross(r, f); // up corrige

    Mat4d V = Mat4d::Identity();
    V(0,0)=r.x; V(0,1)=r.y; V(0,2)=r.z;
    V(1,0)=u.x; V(1,1)=u.y; V(1,2)=u.z;
    V(2,0)=-f.x; V(2,1)=-f.y; V(2,2)=-f.z;
    V(0,3) = -Dot(r, eye); // translation
    V(1,3) = -Dot(u, eye);
    V(2,3) =  Dot(f, eye);
    return V;
}
```

Listing 18: Mat4d — LookAt

0.10.3 Boucle de rendu

```
for (int frame = 0; frame < 10; frame++) {
    // Test 37 : angle different a chaque frame
    double angle = frame * (kPi / 5.0); // 36 degres par frame

    Mat4d rotation = Mat4d::RotateAxis(Vec3d(0,1,0), angle);
    NkImage img(512, 512);
    img.Fill(30, 30, 30); // fond gris

    // Projette et dessine les 12 aretes en rouge
    for (int i = 0; i < 12; i++)
        img.DrawLine(px[a], py[a], px[b], py[b], 255, 0, 0);

    // Test 36 : sauvegarde PPM
    char filename[64];
    snprintf(filename, 64, "frame_%02d.ppm", frame);
    img.SavePPM(filename);
}
```

Listing 19: rasterizer-lookat.cpp — 10 frames

Test	Description	Résultat
32	Cube unitaire $[-0.5, 0.5]^3$	8 coins, 12 arêtes
33	LookAt(eye=$\{0, 1, 3\}$, target=$\{0, 0, 0\}$)	Vue créée
34	Projection perspective fov=60°	Caméra configurée
35	12 arêtes en rouge	DrawLine OK
36	Sauvegarde PPM P6	frame_00.ppm → frame_09.ppm
37	Rotation 36° par frame	Animation OK

Table 5: Résultats des tests 32 à 37

`https://github.com/TAKOUDJOU2371/Nkentseu`

Semaine 4 — Quaternions & Animation SLERP

Objectifs de la semaine

Pourquoi les quaternions ?

Les angles d'Euler souffrent du **gimbal lock** : quand pitch $= \pm 90$, les axes yaw et roll deviennent coplanaires et le système perd un degré de liberté. Les quaternions résolvent ce problème et offrent une interpolation sphérique (SLERP) qui suit le chemin le plus court sur la sphère des rotations S^3.

0.11 Théorie : Quaternions

0.11.1 Définition

Un quaternion $q = w + xi + yj + zk$ est un nombre hypercomplexe avec $i^2 = j^2 = k^2 = ijk = -1$. Un quaternion unitaire ($\|q\| = 1$) représente une rotation de $2\arccos(w)$ radians autour de l'axe (x, y, z).

$$q = \cos\frac{\theta}{2} + \sin\frac{\theta}{2}(n_x i + n_y j + n_z k) \tag{6}$$

0.11.2 Produit de Hamilton

$$q_1 \otimes q_2 = \begin{pmatrix} w_1w_2 - x_1x_2 - y_1y_2 - z_1z_2 \\ w_1x_2 + x_1w_2 + y_1z_2 - z_1y_2 \\ w_1y_2 - x_1z_2 + y_1w_2 + z_1x_2 \\ w_1z_2 + x_1y_2 - y_1x_2 + z_1w_2 \end{pmatrix} \tag{7}$$

0.11.3 SLERP — Spherical Linear Interpolation

$$\mathrm{Slerp}(q_a, q_b, t) = \frac{\sin((1-t)\theta)}{\sin\theta} q_a + \frac{\sin(t\theta)}{\sin\theta} q_b \tag{8}$$

où $\theta = \arccos(q_a \cdot q_b)$.

LERP vs SLERP

Le LERP (interpolation linéaire) sur les quaternions produit une **vitesse angulaire non-uniforme** : le mouvement est plus lent au début et à la fin. Le SLERP garantit une vitesse angulaire constante, ce qui est visuellement correct pour les animations.

0.12 TP1 — Quaternions complets (Tests 41 à 43)

0.12.1 Objectif

Implémenter `Quat` avec `FromAxisAngle`, `Rotate`, `ToMat3`, `FromMat3` (méthode de Shepperd) et vérifier l'inverse.

0.12.2 Implémentation de Quat.h

```
struct Quat {
    double w, x, y, z;

    Quat() : w(1), x(0), y(0), z(0) {} // identite

    double Norm2() const { return w*w+x*x+y*y+z*z; }
    double Norm()  const { return std::sqrt(Norm2()); }

    Quat Normalized() const {
        double n = Norm();
        return {w/n, x/n, y/n, z/n};
    }

    // Conjugue : inverse la rotation
    Quat Conjugate() const { return {w, -x, -y, -z}; }

    // Inverse : pour unitaire, Inverse == Conjugate
    Quat Inverse() const {
        double n2 = Norm2();
        return {w/n2, -x/n2, -y/n2, -z/n2};
    }

    // Produit de Hamilton
    Quat operator*(const Quat& o) const {
        return {
            w*o.w - x*o.x - y*o.y - z*o.z,
            w*o.x + x*o.w + y*o.z - z*o.y,
            w*o.y - x*o.z + y*o.w + z*o.x,
            w*o.z + x*o.y - y*o.x + z*o.w
        };
    }
};

// Depuis axe-angle
inline Quat FromAxisAngle(const Vec3d& axis, double angle) {
    Vec3d n = axis.Normalized();
    double s = std::sin(angle / 2.0);
    double c = std::cos(angle / 2.0);
    return {c, n.x*s, n.y*s, n.z*s};
```

```
}

// Rotation d'un vecteur par un quaternion (version optimisee)
inline Vec3d Rotate(const Quat& q, const Vec3d& v) {
    Vec3d qv = {q.x, q.y, q.z};
    Vec3d uv  = Cross(qv, v);
    Vec3d uuv = Cross(qv, uv);
    return v + (uv * (2.0 * q.w)) + (uuv * 2.0);
}
```

Listing 20: Quat.h — structure complete

0.12.3 Méthode de Shepperd — ToMat3 / FromMat3

```
// Quat vers Mat3
inline Mat3d ToMat3(const Quat& q) {
    double xx=q.x*q.x, yy=q.y*q.y, zz=q.z*q.z;
    double xy=q.x*q.y, xz=q.x*q.z, yz=q.y*q.z;
    double wx=q.w*q.x, wy=q.w*q.y, wz=q.w*q.z;
    Mat3d R;
    R(0,0)=1-2*(yy+zz); R(0,1)=2*(xy-wz); R(0,2)=2*(xz+wy);
    R(1,0)=2*(xy+wz); R(1,1)=1-2*(xx+zz); R(1,2)=2*(yz-wx);
    R(2,0)=2*(xz-wy); R(2,1)=2*(yz+wx); R(2,2)=1-2*(xx+yy);
    return R;
}

// Mat3 vers Quat -- Shepperd (4 cas selon pivot)
inline Quat FromMat3(const Mat3d& R) {
    double trace = R(0,0) + R(1,1) + R(2,2);
    Quat q;
    if (trace > 0) {
        double s = 0.5 / std::sqrt(trace + 1.0);
        q.w = 0.25 / s;
        q.x = (R(2,1) - R(1,2)) * s;
        q.y = (R(0,2) - R(2,0)) * s;
        q.z = (R(1,0) - R(0,1)) * s;
    }
    // ... (cas x, y, z dominants)
    return q.Normalized();
}
```

Listing 21: Quat.h — ToMat3 et FromMat3

0.12.4 Tests (41 à 43)

```
// Test 41 : Rotate(FromAxisAngle(Y,PI/2), {1,0,0}) ~= {0,0,-1}
Quat q = FromAxisAngle(Vec3d(0,1,0), kPi/2.0);
Vec3d r = Rotate(q, Vec3d(1.0, 0.0, 0.0));
assert(approxEq(r.x,  0.0, 1e-9));
assert(approxEq(r.y,  0.0, 1e-9));
assert(approxEq(r.z, -1.0, 1e-9));

// Test 42 : Aller-retour Quat->Mat3->Quat (50 fois)
for (int i = 0; i < 50; i++) {
    Quat q1 = genRandQuat(seed).Normalized();
```

```
    Mat3d m  = ToMat3(q1);
    Quat q2  = FromMat3(m);
    // q1 == q2 ou q1 == -q2 (meme rotation)
    bool same = (q1 == q2) || (q1 == -q2);
    assert(same);
}

// Test 43 : q x   q     = identite
Quat qInv = q.Inverse();
Quat I    = q * qInv;
assert(approxEq(I.w, 1.0, 1e-9));
assert(approxEq(I.x, 0.0, 1e-9));
assert(approxEq(I.y, 0.0, 1e-9));
assert(approxEq(I.z, 0.0, 1e-9));
```

Listing 22: Tests Quaternions

Test	Description	Résultat
41	Rotate(Y,PI/2,{1,0,0}) $\approx$ {0,0,-1}	OK
42	Aller-retour Quat→Mat3→Quat, 50 fois	50/50 OK
43	$q \times q^{-1}$ = identité	OK

Table 6: Résultats des tests 41 à 43

0.13 TP2 — Animation SLERP (Tests 44 à 46)

0.13.1 Objectif

Animer une rotation en 60 frames, comparer SLERP vs LERP, et utiliser le rasteriseur du TP M03 pour afficher le cube.

0.13.2 Implémentation SLERP

```
inline Quat Slerp(Quat a, Quat b, double t) {
    double cosAngle = a.w*b.w + a.x*b.x + a.y*b.y + a.z*b.z;

    // Chemin court : si cosAngle < 0, inverser b
    if (cosAngle < 0.0) {
        b.w = -b.w; b.x = -b.x; b.y = -b.y; b.z = -b.z;
        cosAngle = -cosAngle;
    }

    double k0, k1;
    if (cosAngle > 0.9999) {
        // Angle trop petit -> LERP pour eviter sin(0)/0
        k0 = 1.0 - t;
        k1 = t;
    } else {
        double angle    = std::acos(cosAngle);
        double sinAngle = std::sin(angle);
        k0 = std::sin((1.0 - t) * angle) / sinAngle;
        k1 = std::sin(t * angle) / sinAngle;
    }
    return Quat{k0*a.w+k1*b.w, k0*a.x+k1*b.x,
```

```
                    k0*a.y+k1*b.y, k0*a.z+k1*b.z}.Normalized();
}
```

Listing 23: Quat.h — SLERP

0.13.3 Comparaison SLERP vs LERP

Frame	Angle SLERP (°)	Angle LERP (°)	Différence
0	0.0000	0.0000	0.0000
1	16.0000	15.xxxx	≈ 0.xxxx
4	64.0000	63.xxxx	≈ 0.xxxx
9	144.0000	144.0000	0.0000

Table 7: Test 45 : SLERP vs LERP sur 10 frames

Interprétation visuelle

SLERP (vert) : la vitesse angulaire est constante — le cube tourne de façon uniforme.
LERP (rouge) : la vitesse angulaire est non-uniforme — le cube accélère et ralentit en milieu de parcours.

0.13.4 Tests (44 à 46)

```
// Test 44 : Animation SLERP 60 frames
Quat qa = FromAxisAngle(Vec3d(0,1,0), 0.0);
Quat qb = FromAxisAngle(Vec3d(0,1,0), kPi);
for (int frame = 0; frame < 60; frame++) {
    double t   = (double)frame / 59.0;
    Quat   qs  = Slerp(qa, qb, t);
    Mat4d  Ms  = QuatToMat4(qs);
    NkImage img(512, 512);
    img.Fill(20, 20, 20);
    drawCube(img, view, Ms, 0, 255, 0); // vert
    img.SavePPM("slerp_frame_XX.ppm");
}

// Test 45 : SLERP vs LERP -- angles differents
Quat qs = Slerp(qa, qb, t);
Quat ql = LerpQuat(qa, qb, t);
// SLERP : vitesse uniforme
// LERP  : vitesse non-uniforme

// Test 46 : Cube rotation 60 frames via rasteriseur
Quat qStart = FromAxisAngle(Vec3d(0,1,0), 0.0);
Quat qEnd   = FromAxisAngle(Vec3d(1,1,0).Normalized(), kPi*2.0);
for (int frame = 0; frame < 60; frame++) {
    double t = (double)frame / 59.0;
    Quat   q = Slerp(qStart, qEnd, t);
    Mat4d  M = QuatToMat4(q);
    NkImage img(512, 512);
    img.Fill(15, 15, 30); // fond bleu nuit
```

```
    drawCube(img, view, M, 0, 200, 255);
    img.SavePPM("cube_quat_XX.ppm");
}
```

Listing 24: Tests SLERP et animation

Test	Description	Résultat
44	SLERP 60 frames PPM	slerp_frame_00..59.ppm
45	SLERP vert vs LERP rouge	Différence visible
46	Cube rotation 60 frames	cube_quat_00..59.ppm

Table 8: Résultats des tests 44 à 46

Conclusion

Bilan des travaux pratiques

Ces 4 semaines de travaux pratiques ont permis de construire les fondations mathématiques du pipeline de réalité augmentée.

Sem.	TP	Fichier	Tests
S1	TP1 & TP2	Float.cpp	1–10
S1	TP3	Float.h	11–14
S2	TP1	Vec2d.h	15–19
S2	TP2	Vec3d.h	20–23
S2	TP3	Vec4d.h + NkImage.h	24–27
S3	TP1	Mat4d.h + NkImage.h	28–31
S3	TP2	trs-decomposition.cpp	38–40
S3	TP3	rasterizer-lookat.cpp	32–37
S4	TP1	Quat.h	41–43
S4	TP2	quat-slerp.cpp	44–46
Total			**46 tests**

Table 9: Récapitulatif des 11 TPs

Acquis fondamentaux

- Maîtrise de la représentation IEEE 754 et de l'arithmétique flottante robuste
- Implémentation complète des types vectoriels (Vec2d, Vec3d, Vec4d) avec layout mémoire garanti par `static_assert`
- Matrices 4×4 en column-major avec inverse Gauss-Jordan
- Transformations TRS, LookAt, projection perspective
- Rasteriseur logiciel avec algorithme de Bresenham
- Quaternions unitaires avec SLERP et méthode de Shepperd
- Animation 3D from scratch sans aucune bibliothèque tierce
- Build system Jenga et moteur Nkentseu

Perspectives

Les semaines suivantes s'appuieront sur ces fondations pour développer :

- **Semaine 5** : SVD — Décomposition en Valeurs Singulières
- **Semaine 6** : NkImage — Traitement d'image from scratch
- **Semaine 7+** : Calibration Zhang, détection ArUco, EPnP, pipeline AR complet

Rapport rédigé par KENNE TAKOUDJOU Franck Manuel
AIA4 — ENSPY Yaoundé — Mars 2026